MetaboTrack API: Architectural Design and Analytical Implementation

1. Project Links and Overview

1.1 Project Links

In accordance with the deliverable requirements , the following resources are publicly accessible:

- **GitHub Repository:** MetaboTrack-API GitHub Repository
- **API Documentation (PDF):** MetaboTrack-API - API_Documentation
- **Presentation Slides:** MetaboTrack API - Presentation Slides

1.2 System Overview

The MetaboTrack API is a data-driven web service. The system is designed to manage and interpret a complex physiological dataset containing 15,000+ daily metabolic records (Parashar, 2026).

Unlike conventional "Data Sink" applications that merely provide basic CRUD functionality, MetaboTrack is engineered as an Analytical Insight Engine. It addresses the challenge of large-scale health data management by providing sophisticated endpoints for:

1. **Macro-level Population Insights:** Utilizing advanced SQL aggregations to identify trends in metabolic efficiency and cardiovascular health across the entire dataset.
2. **Predictive Simulation:** Implementing a deterministic, rule-based engine that allows users to project future health outcomes based on hypothetical lifestyle adjustments.
3. **Performance Ranking:** Computing real-time, individual percentile rankings to enhance user engagement through data-driven feedback.

The system adopts a robust, layered architecture utilizing Spring Boot 3.x and MyBatis-Plus, ensuring high scalability and maintainability while maintaining a focus

on performance optimization for large-sample data analysis.

2. Technology Stack & Architectural Justification

2.1 Technology Stack Justification

To ensure the MetaboTrack API is robust, scalable, and capable of processing over 15,000 physiological records, the following technology stack was deliberately selected:

- **Java 17 & Spring Boot 3:** Chosen for its robust object-oriented paradigm, strong type safety, and the powerful Inversion of Control (IoC) container. Spring Boot significantly accelerates backend development by providing embedded web servers and auto-configuration, allowing the focus to remain on complex business logic rather than boilerplate setup.
- **MySQL 8:** A relational database is the optimal choice for highly structured, tabular

physiological data (e.g., steps, heart rate, calories). Furthermore, strategic B-Tree indexing on columns like `user_id` and `calorie_efficiency` was implemented to reduce query time complexity from $O(N)$ (full table scans) to $O(\log N)$, ensuring rapid pagination and aggregation.

- **MyBatis-Plus & Hybrid SQL Strategy:** A conscious hybrid approach was adopted for data access. For standard CRUD operations, MyBatis-Plus's `LambdaQueryWrapper` was utilized to guarantee compile-time type safety and prevent SQL injection. However, for advanced analytical endpoints, ORM abstractions were intentionally bypassed in favor of native `@Select` SQL queries to maximize the database engine's aggregation performance.

```
Page<DailyMetabolicRecord> resultPage = lambdaQuery()
        .eq( condition: recordPageQueryDTO.getUserId() != null, DailyMetabolicRecord::getUserId, recordPageQueryDTO.getUserId())
        .eq( condition: recordPageQueryDTO.getCalorieEfficiency() != null, DailyMetabolicRecord::getCalorieEfficiency, recordPageQueryDTO.getCalorieEfficiency())
        .orderByDesc(DailyMetabolicRecord::getEfficiencyScore)
        .page(page);
```

```
@Select("SELECT COUNT(*) FROM (" +
        "  SELECT user_id, AVG(efficiency_score) as avg_score " +
        "  FROM daily_metabolic_record " +
        "  GROUP BY user_id " +
        "  HAVING avg_score < #{targetScore}" +
        ") AS subquery")
Long countUsersBelowScore(@Param("targetScore") Double targetScore);
```

Figure 1. *Screenshots of MP's LambdaQuery and @Select SQL query*

2.2 Architectural Design Decisions

The application strictly adheres to the Controller-Service-Mapper layered architecture to guarantee the Separation of Concerns (SoC). Beyond this standard structure, two critical architectural patterns were implemented to elevate the system to an enterprise-grade standard:

1. Input Boundary Control via DTO/VO Pattern

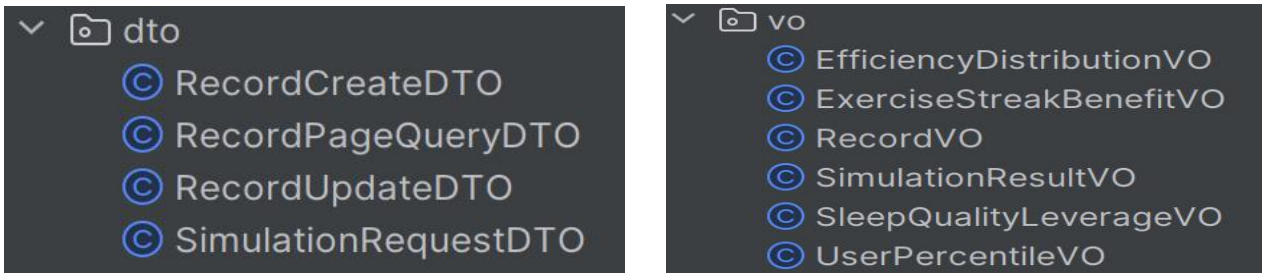

Figure 2. *Screenshots of DTOs and VOs*

Exposing internal database entities (e.g., `DailyMetabolicRecord`) directly to REST controllers is a well-known anti-pattern that introduces severe security vulnerabilities, such as over-posting attacks (e.g., malicious users injecting arbitrary `user_ids` during record updates). To enforce strict input boundaries, Data Transfer Objects (DTOs) such as `RecordCreateDTO` and `RecordUpdateDTO` were implemented to act as an isolation layer. Similarly, Value Objects (VOs) were designed to shape the analytical outputs precisely for frontend visualization requirements.

2. Database Engine Offloading for Macro-Insights

When dealing with a dataset of 15,000+ rows, performing data aggregations (e.g., calculating population percentile distributions or sleep quality leverage) within the Java JVM using loops leads to severe memory overhead and risks `OutOfMemoryError`. To mitigate this, complex analytical logic was pushed down to the database layer. By utilizing advanced SQL features like `GROUP BY`,

nested subqueries, and `CASE WHEN` for Data Binning (e.g., categorizing continuous sleep hours into discrete buckets), the application achieves an $O(1)$ memory space complexity, transferring only the final aggregated metrics across the network.

3. Dynamic Simulation and Predictive Engine

The core innovation of the MetaboTrack API is the Dynamic Predictor and Simulator (`/api/analytics/simulator`). While conventional endpoints describe historical states, this engine functions as a proactive tool for behavioral health modeling, allowing users to project future health outcomes based on hypothetical lifestyle adjustments.

3.1 Deterministic Rule-Based Design

The simulator is architected as an in-memory Deterministic Rule-Based Engine. By strictly implementing the dataset author's physiological heuristics(Parashar, 2026), it accurately models the interplay between lifestyle variables and metabolic outcomes. Executing this logic entirely within Java RAM offers two critical advantages:

1. **Latency Optimization:** Simulations are computed in $O(1)$ time, enabling instantaneous "what-if" feedback for interactive UI components without the latency of database I/O.
2. **Prescriptive Value:** The logic is isolated from the persistence layer, allowing the system to project hypothetical scenarios that do not yet exist in the database, thereby guiding users toward healthier habits.

2.1 Explicit Calculation Heuristics & Formulas

Phase	Metric	Formula / Adjustment Rule
1. Base Signal	Core Efficiency	$Base = \frac{Calories}{Steps + (20 \times ActiveMinutes)}$
2. Linear Weights	Muscle Mass	$+0.3$
	Body Fat	-0.2
	Hydration	$+0.05$ per liter
	Sleep Hours	$+0.05$ per hour
1. Non-Linear Multipliers	Resting HR Scaler	$Score = Score \times \left(\frac{80}{RestingHR}\right)$
	Average HR Scaler	$Score = Score \times \left(\frac{120}{AverageHR}\right)$
2. Consistency & Penalties	Habit Bonus	$+3\%$ per continuous day ($Score \times 1.03$)
	Adaptation Jump	$+10\%$ flat jump upon crossing the 5-day streak
	Over-training Drop	-15% penalty for workouts > 6 days/week
	Exhaustion Penalty	-10% penalty if streak ≥ 6AND sleep < 6 hours
	Severe Deprivation	-25% flat drop if sleep < 5hours
3. Boundary Enforcement	Numerical Limits	Math.min(10.0, Math.max(0.0, Score))

Table 1. Efficiency Calculation Rules

To achieve high physiological fidelity, the engine translates the dataset's underlying logic into a strict, multi-phase mathematical model. The complete calculation pipeline is defined in the heuristic table above

The continuous score is finally mapped to categorical tiers: High Efficiency (≥ 7.0), Moderate ($4.0 - 6.9$), and Low Efficiency (< 4.0). This deterministic approach demonstrates a deep understanding of domain-specific business logic beyond simple data storage.

```
// 1. The Base Signal
// efficiency = calories_burned / (steps_per_day + 20 * active_minutes)
double denominator = req.getTargetSteps() + (20.0 * req.getTargetActiveMinutes());
double baseSignal = denominator > 0 ? (req.getTargetCaloriesBurned() / denominator) * 100.0 : 0.0;
double score = baseSignal;
feedback.add("Base signal calculated from movement-to-calorie ratio.");

// 2. Linear Weights
// Hydration: +0.05 weight | Sleep: +0.05 weight
score += (req.getTargetHydration() * 0.05);
score += (req.getTargetSleepHours() * 0.05);
```

Figure 3. *Efficiency calculation rule code (partial)*

4. Testing Approach & Error Handling

The reliability and usability of the MetaboTrack API are ensured through a centralized error-handling architecture and an interactive documentation-driven testing strategy.

1. **Global Exception Handling and Standardized Responses:** To maintain a robust and secure API contract, scattered `try-catch` blocks were avoided. Instead, a global interceptor utilizing `@RestControllerAdvice` was implemented. This mechanism catches both custom `BusinessException` instances (e.g., "Record Not Found") and unexpected runtime errors globally. It guarantees that the client always receives a predictable, standardized JSON wrapper (`Result<T>`) containing appropriate HTTP status codes and actionable error messages, strictly preventing internal Java stack traces from leaking to the frontend.
2. **Interactive Documentation-Driven Testing:** Springdoc OpenAPI (Swagger 3) was integrated not merely as static documentation, but as the primary interactive testing suite. By leveraging the `@Schema(example="...")` annotation across all DTOs and Controllers, every endpoint is pre-populated with valid JSON payloads and path variables. This design choice drastically lowers the barrier to entry, allowing developers to conduct immediate, successful integration tests directly from the browser without requiring external tools like Postman.

5. Limitations & Future Development

While the API efficiently fulfills the coursework requirements and successfully manages the 15,000+ record dataset, certain architectural limitations exist that would require mitigation before a production-grade release.

1. **Security and Authentication Deficiencies:** Currently, the API endpoints rely on raw `userId` parameters passed directly in the request path or body. This presents a critical security vulnerability known as Insecure Direct Object Reference (IDOR), where malicious users could manipulate parameters to access or alter others' data.
   - **Future mitigation** requires the implementation of a robust authentication framework, such as JWT (JSON Web Tokens) via Spring Security, ensuring that user identity is securely

extracted from the HTTP Authorization header rather than explicit parameters.

2. **High-Concurrency Database Bottlenecks:** Although offloading macro-analytics to the database resolves JVM memory constraints, repeatedly executing complex nested subqueries (e.g., the percentile ranking engine) under high concurrent traffic would inevitably strain the MySQL CPU.
   - **Future development** would introduce an in-memory caching layer using Redis. Expensive macro-analytics and leaderboards would be cached and scheduled to refresh periodically (e.g., hourly) using Spring Cache, drastically reducing direct database I/O and ensuring sub-millisecond response times.

6. Generative AI Declaration & Usage Analysis

In accordance with the "Green Light Assessment" policy, I formally declare the use of Generative AI tools (Google Gemini 3.1 Pro) throughout the lifecycle of this coursework. Full conversation logs are attached as supplementary material.

My engagement with Generative AI involved architectural-level discussion and technical implementation assistance, demonstrating creative application across four key domains:

1. **Construction of the Data Ingestion Pipeline**

   I utilized GenAI to assist in developing an efficient Python script to populate the database with the 15,000-record dataset. We discussed optimizing the I/O process, which led to the implementation of an in-memory Enum Mapping strategy. This approach converted categorical string labels into integers prior to database insertion, significantly reducing the B-Tree index footprint and accelerating future query speeds.

2. **Optimization of Backend SQL Aggregation & Database Offloading**

   A significant portion of my dialogue involved debating the space complexity trade-offs of macro-analytics. We explored the performance implications of fetching records into the Java JVM versus pushing computation down to the MySQL engine. This led to the architectural decision to implement In-Database Aggregations via nested subqueries, ensuring an $O(1)$ memory footprint for the Spring Boot application and preventing `OutOfMemoryError` under high concurrency.

3. **Mathematical Dissection of the Simulator Algorithm**

   The development of the `/simulator` endpoint required a rigorous mathematical translation of the physiological heuristics explicitly provided by the dataset author. I utilized GenAI to help dissect these abstract behavioral descriptions into a deterministic mathematical pipeline. This collaborative analysis ensured that the Java-based Rule Engine maintained perfect algorithmic fidelity to the author's specified generation logic while remaining computationally efficient.

4. **Professional API Documentation Engineering**

   To ensure the API deliverables met publication-quality standards, I collaborated with GenAI to craft precise, industry-standard descriptions and structural schemas for the Springdoc OpenAPI (Swagger 3) integration. The AI assisted in formulating complex `@Schema(example="...")` JSON payloads for various DTOs, lowering the barrier to entry and enabling interactive, documentation-driven testing directly from the browser.

References

[1]. Parashar, M., 2026. *Close to realistic Calorie efficiency dataset*. [dataset] Kaggle. Available at: https://www.kaggle.com/datasets/parasharmanu/close-to-realistic-calorie-efficiency-dataset [Accessed 10 April 2026].